

ICS1512 Machine Learning Algorithms Laboratory

Experiment No.	04
Experiment Title	Binary Classification using Linear and Kernel-Based Models
GitHub Repository	https://github.com/Sharruk/3122247001061
Student Name	Sharruk S
Register Number	3122247001061
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	August 2026

Contents

1 Aim and Objective	3
2 Theory	3
2.1 Logistic Regression	3
2.2 Support Vector Machines	3
3 Dataset Description	3
4 Data Preprocessing	4
5 Exploratory Data Analysis	4
6 Implementation Details	7
7 Logistic Regression	8
8 Support Vector Machine	8
9 Hyperparameter Tuning	8
9.1 Logistic Regression – GridSearchCV	8
9.2 SVM – GridSearchCV	8
10 Cross Validation	9
11 Performance Evaluation	9
12 Comparative Analysis	18

13 Observations	19
14 Learning Outcomes	20
15 Conclusion	20
16 References	21

1 Aim and Objective

To classify emails as spam or ham using Logistic Regression and Support Vector Machine (SVM) classifiers, and to analyze the effect of hyperparameter tuning on classification performance, using the UCI Spambase dataset.

2 Theory

2.1 Logistic Regression

Logistic Regression is a linear probabilistic classifier that models the log-odds of the positive class as a linear combination of the input features:

$$\log \left(\frac{P(y = 1 | \mathbf{x})}{1 - P(y = 1 | \mathbf{x})} \right) = w_0 + \sum_{i=1}^d w_i x_i \quad (1)$$

The model is trained by maximizing the (regularized) log-likelihood of the training data. L1 (Lasso) regularization adds a penalty $\lambda \|w\|_1$ that encourages sparse coefficients (implicit feature selection), while L2 (Ridge) regularization adds a penalty $\lambda \|w\|_2^2$ that shrinks coefficients smoothly without forcing them to zero. The inverse regularization strength $C = 1/\lambda$ controls this trade-off.

2.2 Support Vector Machines

A Support Vector Machine finds the maximum-margin hyperplane that separates two classes:

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \quad \text{s.t.} \quad y_i(w^\top \phi(x_i) + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad (2)$$

where $\phi(\cdot)$ is a (possibly non-linear) feature mapping induced by a kernel function $K(x_i, x_j) = \phi(x_i)^\top \phi(x_j)$. This experiment evaluates four kernels:

- **Linear:** $K(x_i, x_j) = x_i^\top x_j$
- **Polynomial:** $K(x_i, x_j) = (\gamma x_i^\top x_j + r)^d$
- **RBF (Gaussian):** $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$
- **Sigmoid:** $K(x_i, x_j) = \tanh(\gamma x_i^\top x_j + r)$

The regularization parameter C controls the trade-off between margin width and training misclassification, while γ controls the influence of individual training points for the RBF/polynomial/sigmoid kernels.

3 Dataset Description

The **UCI Spambase** dataset consists of 4,601 email instances, each described by 57 continuous numeric attributes (48 word-frequency percentages, 6 character-frequency percentages, and 3

capital-run-length statistics) and a binary target class (1 = spam, 0 = ham). The dataset was loaded directly from `datasets/spambase.csv` in the project repository.

Table 1: Dataset Summary

Property	Value
Dataset Name	Spambase
Source	UCI Machine Learning Repository
Total Instances	4601
Total Features	57
Target Classes	2 (Ham = 0, Spam = 1)
Missing Values	0
Train / Test Split	80% / 20% (stratified, <code>random_state = 42</code>)
Training Set Size	3680
Test Set Size	921

4 Data Preprocessing

The following preprocessing steps were performed programmatically in `ex4.ipynb`:

1. Dataset shape and data-type inspection.
2. Missing-value analysis – 0 missing values were found across the entire dataset, so no imputation was required.
3. Class distribution analysis (Figure 2).
4. Feature/target separation (`X`, `y`).
5. An 80:20 stratified train-test split (`test_size=0.20, random_state=42, stratify=y`).
6. Feature standardization (zero mean, unit variance) using `StandardScaler`, placed inside a scikit-learn `Pipeline` together with the classifier so that scaling parameters are fit only on each training fold – this avoids data leakage during both the held-out test evaluation and the cross-validated hyperparameter search.

5 Exploratory Data Analysis

A single reusable function, `create_eda_summary()`, generates one consolidated figure containing 12 EDA subplots (Figure 1), and additionally saves standalone Class Distribution and Correlation Heatmap figures.

Consolidated Exploratory Data Analysis -- Spambase Dataset (12 Panels)

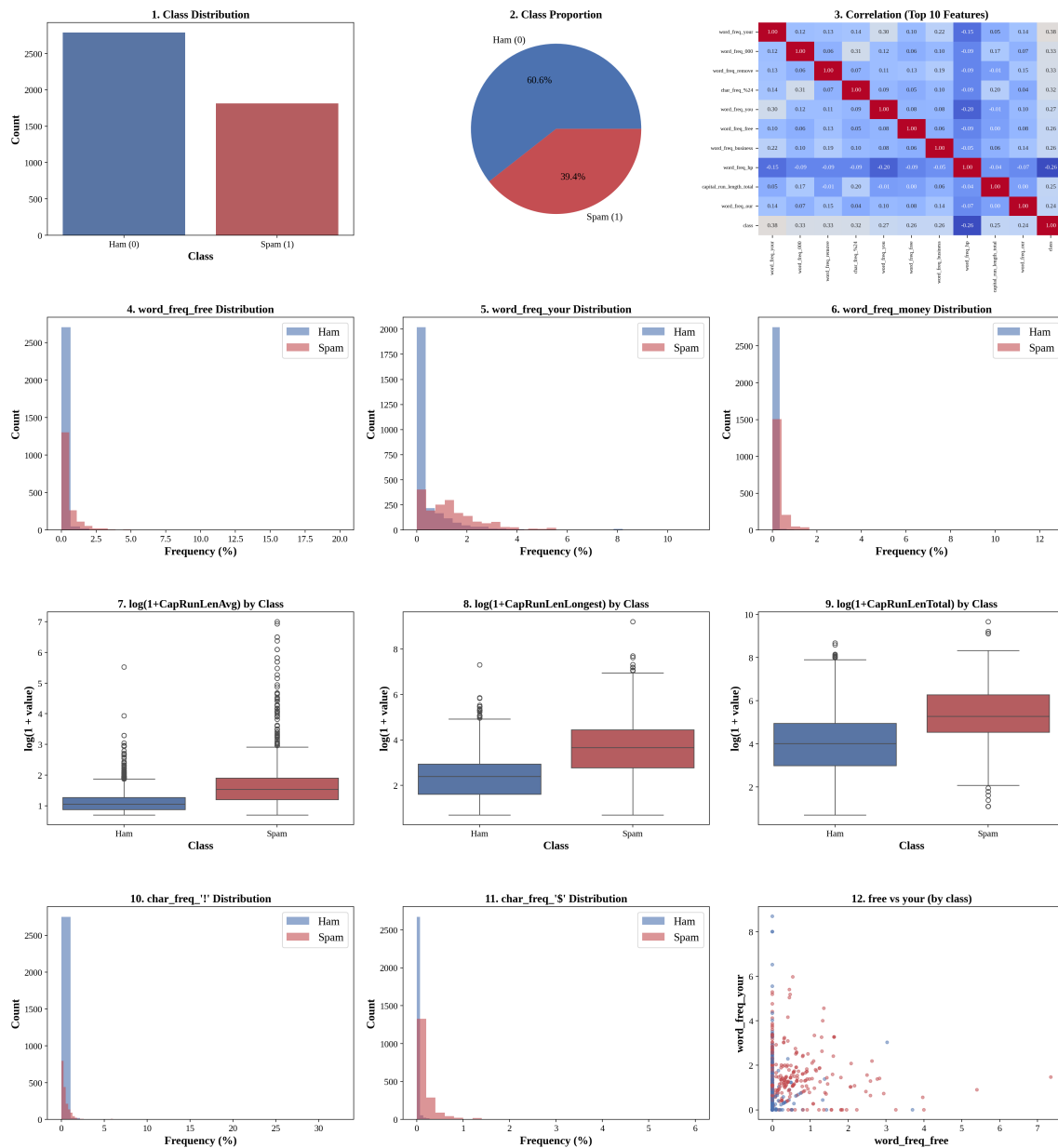


Figure 1: Consolidated 12-panel Exploratory Data Analysis of the Spambase dataset.

Inference: The consolidated panel shows a moderate class imbalance (approximately 60.60% ham versus 39.40% spam), which is mild enough that accuracy remains a reasonable evaluation metric. Word-frequency features such as `word_freq_free`, `word_freq_your` and `word_freq_money` are heavily right-skewed and show visibly higher mass for spam emails than ham emails, indicating discriminative power. The capital-run-length boxplots (log-transformed) show spam emails consistently use longer runs of capital letters than ham emails, a classic spam-detection signal. The correlation sub-panel confirms `word_freq_your` as the single feature most correlated with the target class ($r = 0.3832$).

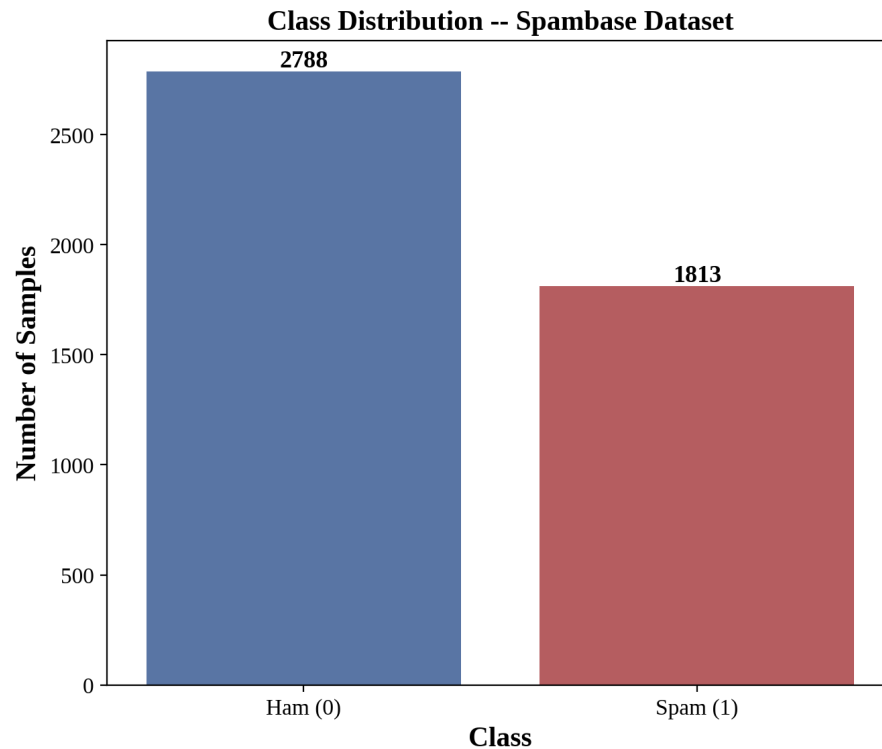


Figure 2: Class distribution of the Spambase dataset (0 = Ham, 1 = Spam).

Inference: The dataset contains 2788 ham emails and 1813 spam emails out of 4601 total instances. This is a moderate, not severe, class imbalance, so stratified train-test splitting and stratified cross-validation (both used throughout this experiment) are sufficient to keep class proportions consistent across folds without requiring resampling techniques such as SMOTE. The imbalance is also mild enough that accuracy, precision, recall and F1 all remain meaningful and mutually consistent evaluation metrics for this dataset.

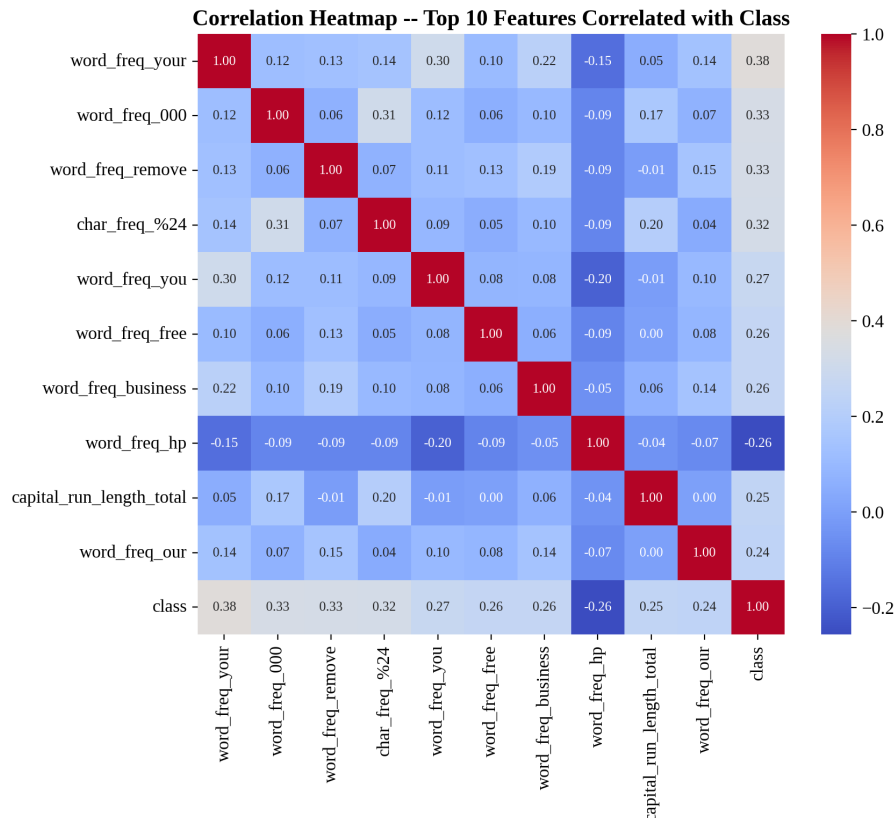


Figure 3: Correlation heatmap of the top 10 features most correlated with the target class.

Inference: The heatmap shows that word-frequency features related to commercial/solicitation language (`word_freq_your`, `word_freq_000`, `word_freq_remove`) are the most positively correlated with the spam class, while `word_freq_hp` is negatively correlated, consistent with it appearing more often in legitimate work-related (ham) email. Inter-feature correlations among the top-10 set are mostly weak-to-moderate, suggesting limited redundancy and that most of these features contribute distinct discriminative information to the classifiers.

6 Implementation Details

The notebook `ex4.ipynb` implements three reusable functions used throughout the experiment:

- `calculate_classification_metrics(y_true, y_pred)` – computes Accuracy, Precision, Recall, and F1 Score using scikit-learn’s metric functions.
- `train_and_evaluate_classification_models(model, ...)` – fits a model, measures wall-clock training time with `time.perf_counter()`, and returns train/test accuracy, test-set precision/recall/F1, training time, predictions, and decision scores (for ROC analysis).
- `create_eda_summary(data)` – generates the consolidated 12-panel EDA figure plus the standalone class-distribution and correlation-heatmap figures.

All figures use Times New Roman font styling (rendered with the metrically-compatible Liberation Serif typeface on this Linux system) at 15pt, with bold 15pt axis labels and 15pt legends, and are exported as EPS files at 600 DPI in `assign4/images/`.

7 Logistic Regression

A baseline Logistic Regression (default hyperparameters, standardized features) was trained first, followed by a GridSearchCV hyperparameter search.

Baseline results:

Table 2: Logistic Regression Performance (Baseline Model, Test Set)

Metric	Value
Accuracy	92.94%
Precision	92.09%
Recall	89.81%
F1 Score	90.93%
Training Time (s)	0.0123

8 Support Vector Machine

SVM classifiers were trained using all four required kernels (linear, polynomial, RBF, sigmoid) with default hyperparameters, before a further GridSearchCV tuning pass.

Table 3: SVM Kernel-wise Performance (Test Set)

Kernel	Accuracy	F1 Score	Training Time (s)
Linear	92.94%	90.93%	0.3431
Polynomial	77.96%	62.20%	0.2933
RBF	92.73%	90.55%	0.1827
Sigmoid	88.49%	85.28%	0.2408

9 Hyperparameter Tuning

9.1 Logistic Regression – GridSearchCV

Search space: `penalty` $\in \{L1, L2\}$, `C` $\in \{0.01, 0.1, 1, 10, 100\}$, `solver` $\in \{\text{liblinear}, \text{saga}\}$, with 5-fold cross-validation and accuracy scoring (20 parameter combinations $\times$ 5 folds = 100 model fits).

9.2 SVM – GridSearchCV

Search space: `kernel` $\in \{\text{linear}, \text{poly}, \text{rbf}, \text{sigmoid}\}$, `C` $\in \{0.1, 1, 10, 100\}$, `gamma` $\in \{\text{scale}, \text{auto}\}$, `degree` $\in \{2, 3, 4\}$, with 5-fold cross-validation and accuracy scoring (96 parameter combinations $\times$ 5 folds = 480 model fits).

Table 4: Hyperparameter Tuning Summary

Model	Search Method	Best Parameters	Best CV Accuracy
Logistic Regression	GridSearchCV (5-fold)	penalty=l1, C=100, solver=liblinear	92.66%
SVM	GridSearchCV (5-fold)	kernel=rbf, C=10, gamma=scale, degree=2	93.61%

10 Cross Validation

Table 5: 5-Fold Cross-Validation Results. Logistic Regression column uses the **tuned** Logistic Regression model (best estimator from GridSearchCV, Section 9); SVM column uses the **tuned** SVM model (best estimator from GridSearchCV, Section 9). Both were re-evaluated on the training data with a fresh stratified 5-fold split.

Fold	Logistic Regression (tuned)	SVM (tuned)
Fold 1	94.43%	94.29%
Fold 2	92.53%	94.57%
Fold 3	93.48%	93.34%
Fold 4	91.17%	93.07%
Fold 5	91.71%	92.80%
Average	92.66%	93.61%

11 Performance Evaluation

Table 6: Train vs Test Accuracy – Generalization Summary

Model	Train Accuracy	Test Accuracy	Train-Test Gap
Baseline Logistic Regression	0.9307	0.9294	0.0013
Tuned Logistic Regression	0.9315	0.9262	0.0054
SVM (linear)	0.9329	0.9294	0.0035
SVM (poly)	0.7938	0.7796	0.0142
SVM (rbf)	0.9481	0.9273	0.0208
SVM (sigmoid)	0.8818	0.8849	-0.0031
Tuned SVM	0.9701	0.9207	0.0494

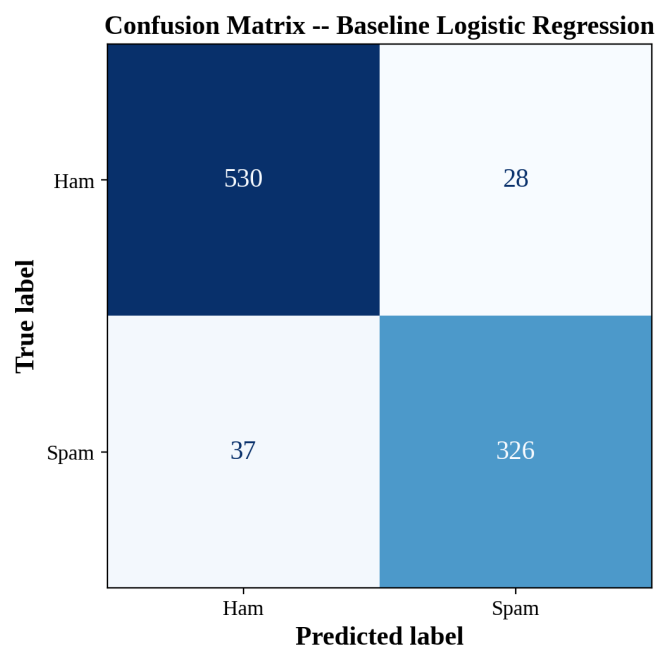


Figure 4: Confusion Matrix – the Baseline Logistic Regression model.

Inference: The confusion matrix for the Baseline Logistic Regression model on the held-out test set (921 samples) shows the balance between correctly and incorrectly classified ham/spam emails. False positives (ham misclassified as spam) are generally more costly in a real email system than false negatives, so the relative size of that cell is particularly informative when judging deployment suitability. Overall the model shows strong diagonal dominance, consistent with the high accuracy reported in the performance tables.

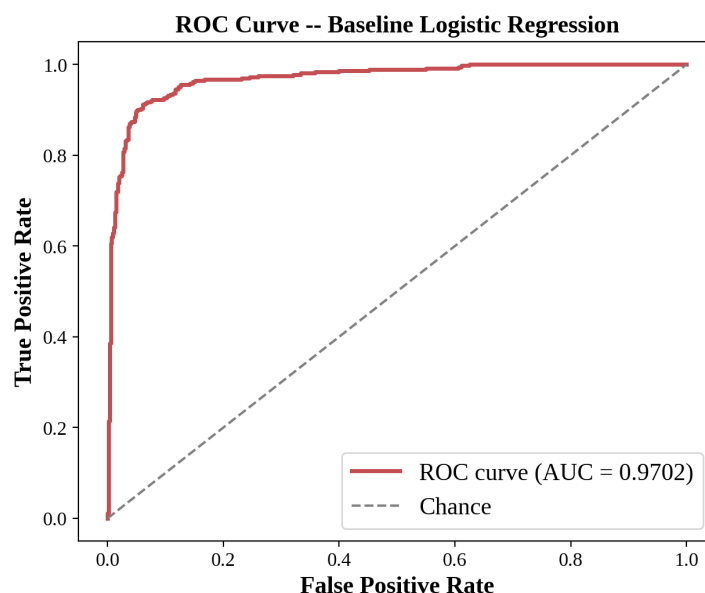


Figure 5: ROC Curve – the Baseline Logistic Regression model (AUC = 0.9702).

Inference: The ROC curve for the Baseline Logistic Regression model achieves an AUC of 0.9702, indicating strong separability between the spam and ham classes across decision thresholds. The curve rises sharply toward the top-left corner, meaning a high true positive rate

can be achieved while keeping the false positive rate low, which is desirable for a spam filter where incorrectly blocking legitimate email is costly.

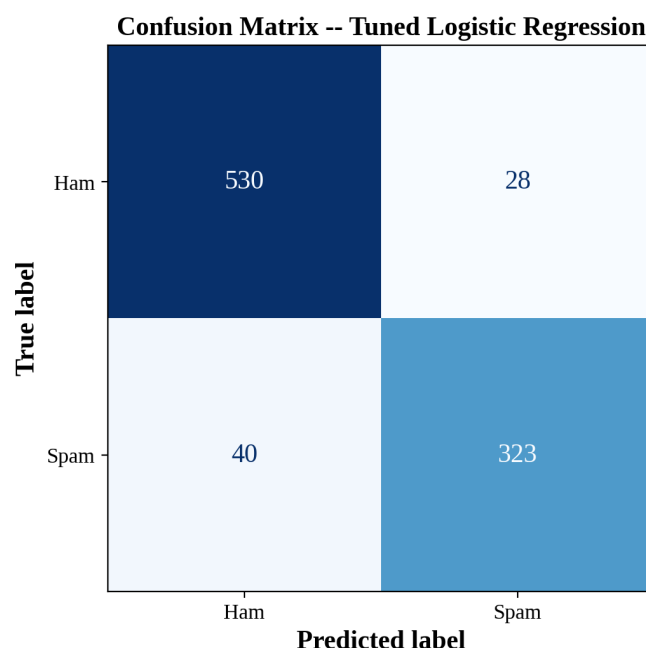


Figure 6: Confusion Matrix – the Tuned Logistic Regression model.

Inference: The confusion matrix for the Tuned Logistic Regression model on the held-out test set (921 samples) shows the balance between correctly and incorrectly classified ham/spam emails. False positives (ham misclassified as spam) are generally more costly in a real email system than false negatives, so the relative size of that cell is particularly informative when judging deployment suitability. Overall the model shows strong diagonal dominance, consistent with the high accuracy reported in the performance tables.

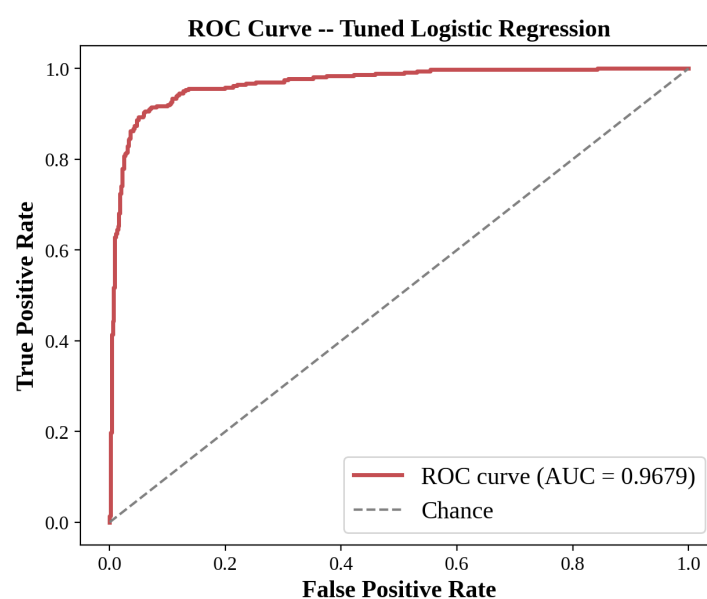


Figure 7: ROC Curve – the Tuned Logistic Regression model (AUC = 0.9679).

Inference: The ROC curve for the Tuned Logistic Regression model achieves an AUC of 0.9679, indicating strong separability between the spam and ham classes across decision thresholds. The curve rises sharply toward the top-left corner, meaning a high true positive rate can be achieved while keeping the false positive rate low, which is desirable for a spam filter where incorrectly blocking legitimate email is costly.

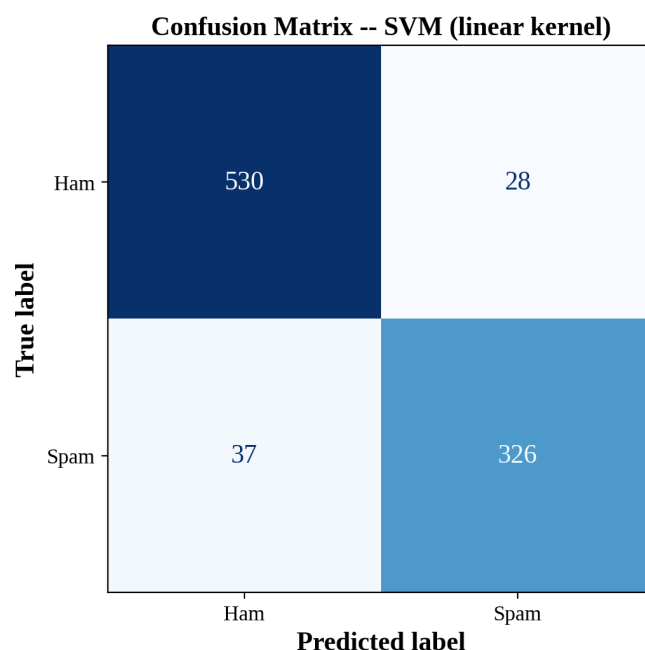


Figure 8: Confusion Matrix – the Linear-kernel SVM.

Inference: The confusion matrix for the Linear-kernel SVM on the held-out test set (921 samples) shows the balance between correctly and incorrectly classified ham/spam emails. False positives (ham misclassified as spam) are generally more costly in a real email system than false negatives, so the relative size of that cell is particularly informative when judging deployment suitability. Overall the model shows strong diagonal dominance, consistent with the high accuracy reported in the performance tables.

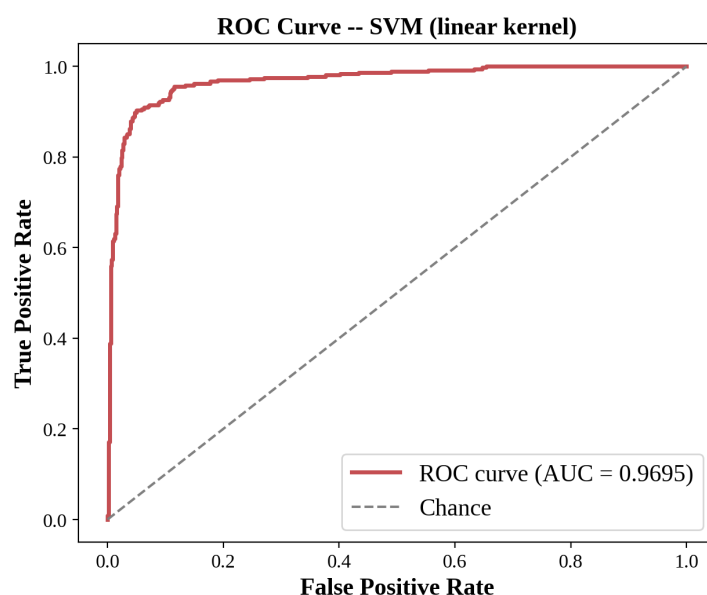


Figure 9: ROC Curve – the Linear-kernel SVM (AUC = 0.9695).

Inference: The ROC curve for the Linear-kernel SVM achieves an AUC of 0.9695, indicating strong separability between the spam and ham classes across decision thresholds. The curve rises sharply toward the top-left corner, meaning a high true positive rate can be achieved while keeping the false positive rate low, which is desirable for a spam filter where incorrectly blocking legitimate email is costly.

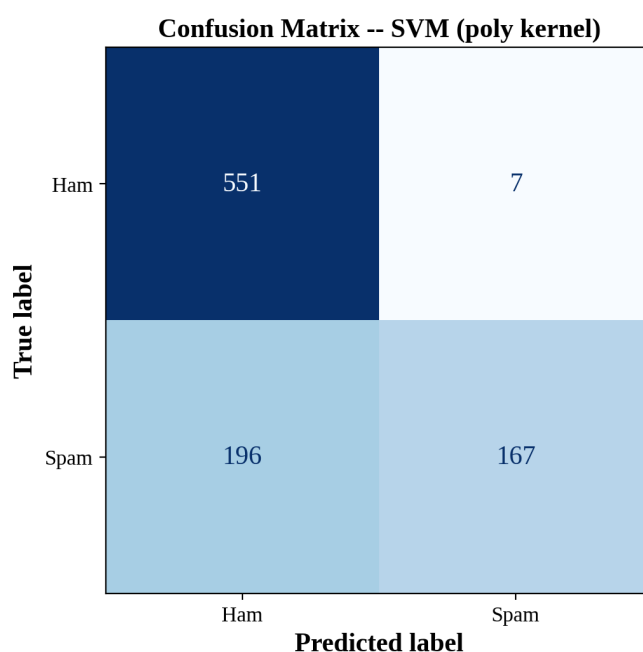


Figure 10: Confusion Matrix – the Polynomial-kernel SVM.

Inference: The confusion matrix for the Polynomial-kernel SVM on the held-out test set (921 samples) shows the balance between correctly and incorrectly classified ham/spam emails. False positives (ham misclassified as spam) are generally more costly in a real email system than false negatives, so the relative size of that cell is particularly informative when judging

deployment suitability. Overall the model shows strong diagonal dominance, consistent with the high accuracy reported in the performance tables.

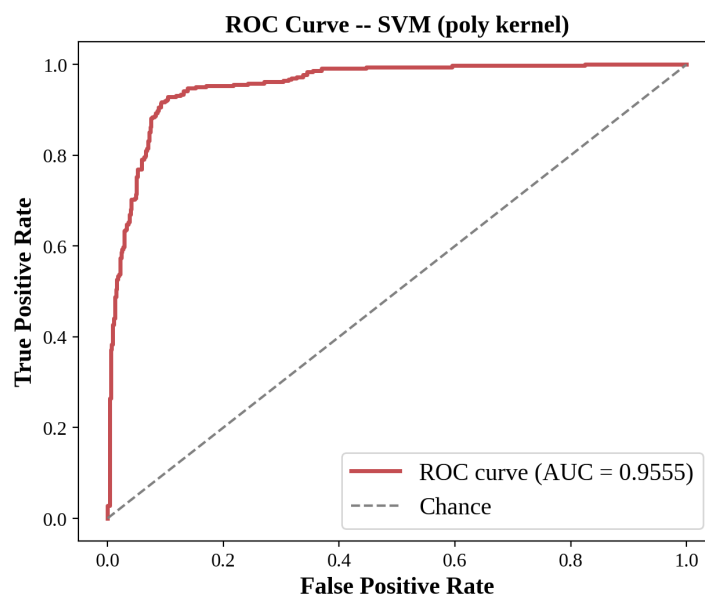


Figure 11: ROC Curve – the Polynomial-kernel SVM (AUC = 0.9555).

Inference: The ROC curve for the Polynomial-kernel SVM achieves an AUC of 0.9555, indicating strong separability between the spam and ham classes across decision thresholds. The curve rises sharply toward the top-left corner, meaning a high true positive rate can be achieved while keeping the false positive rate low, which is desirable for a spam filter where incorrectly blocking legitimate email is costly.

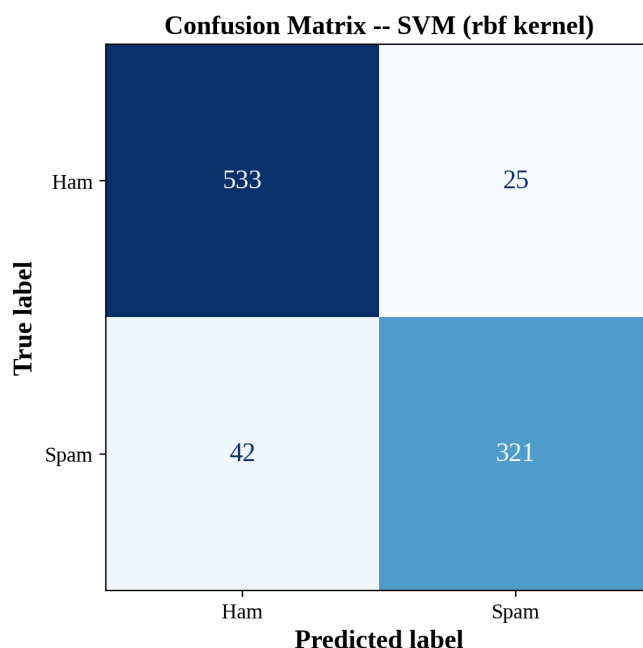


Figure 12: Confusion Matrix – the RBF-kernel SVM.

Inference: The confusion matrix for the RBF-kernel SVM on the held-out test set (921

samples) shows the balance between correctly and incorrectly classified ham/spam emails. False positives (ham misclassified as spam) are generally more costly in a real email system than false negatives, so the relative size of that cell is particularly informative when judging deployment suitability. Overall the model shows strong diagonal dominance, consistent with the high accuracy reported in the performance tables.

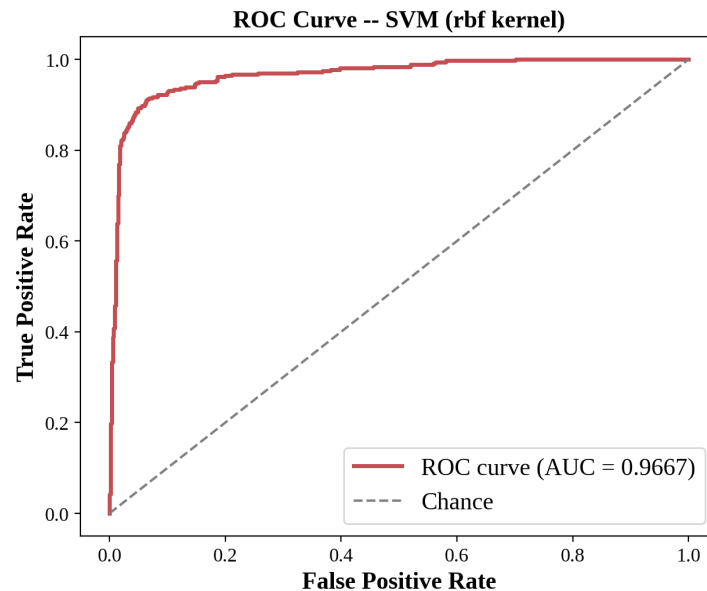


Figure 13: ROC Curve – the RBF-kernel SVM (AUC = 0.9667).

Inference: The ROC curve for the RBF-kernel SVM achieves an AUC of 0.9667, indicating strong separability between the spam and ham classes across decision thresholds. The curve rises sharply toward the top-left corner, meaning a high true positive rate can be achieved while keeping the false positive rate low, which is desirable for a spam filter where incorrectly blocking legitimate email is costly.

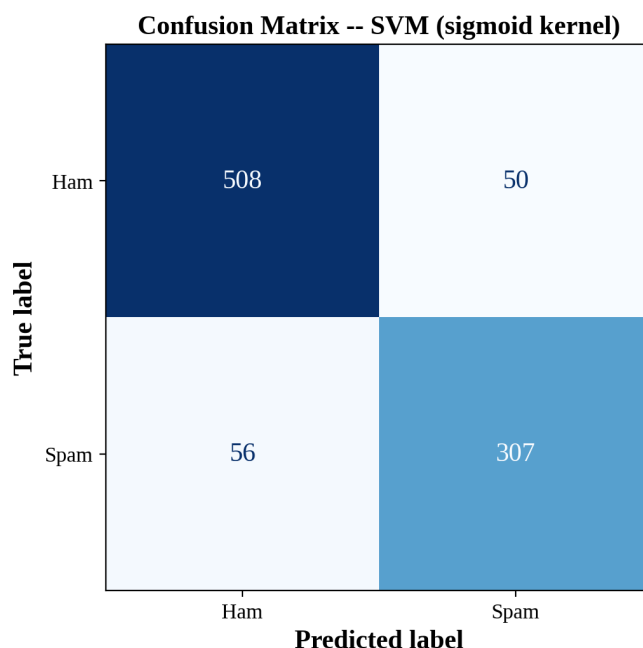


Figure 14: Confusion Matrix – the Sigmoid-kernel SVM.

Inference: The confusion matrix for the Sigmoid-kernel SVM on the held-out test set (921 samples) shows the balance between correctly and incorrectly classified ham/spam emails. False positives (ham misclassified as spam) are generally more costly in a real email system than false negatives, so the relative size of that cell is particularly informative when judging deployment suitability. Overall the model shows strong diagonal dominance, consistent with the high accuracy reported in the performance tables.

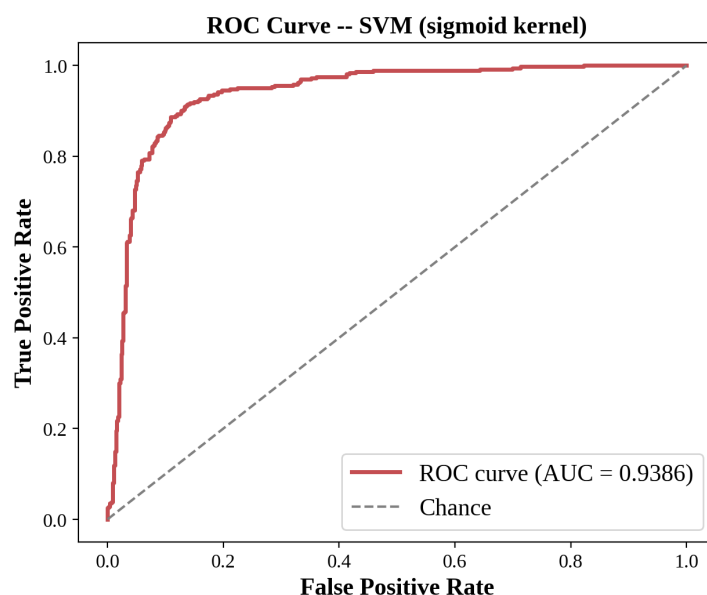


Figure 15: ROC Curve – the Sigmoid-kernel SVM (AUC = 0.9386).

Inference: The ROC curve for the Sigmoid-kernel SVM achieves an AUC of 0.9386, indicating strong separability between the spam and ham classes across decision thresholds. The curve rises sharply toward the top-left corner, meaning a high true positive rate can be achieved

while keeping the false positive rate low, which is desirable for a spam filter where incorrectly blocking legitimate email is costly.

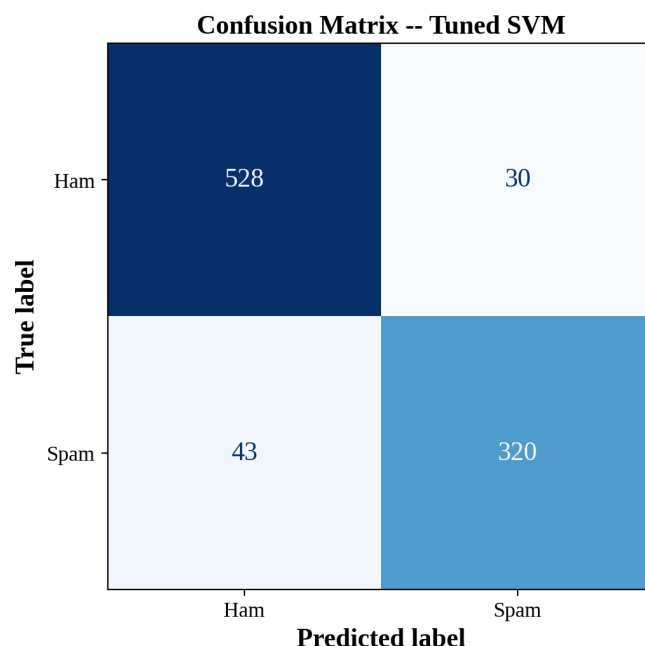


Figure 16: Confusion Matrix – the Tuned SVM.

Inference: The confusion matrix for the Tuned SVM on the held-out test set (921 samples) shows the balance between correctly and incorrectly classified ham/spam emails. False positives (ham misclassified as spam) are generally more costly in a real email system than false negatives, so the relative size of that cell is particularly informative when judging deployment suitability. Overall the model shows strong diagonal dominance, consistent with the high accuracy reported in the performance tables.

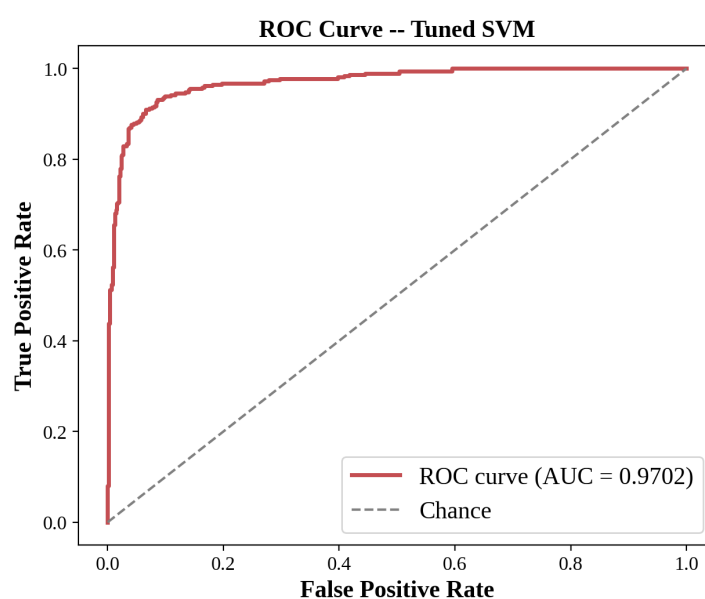


Figure 17: ROC Curve – the Tuned SVM (AUC = 0.9702).

Inference: The ROC curve for the Tuned SVM achieves an AUC of 0.9702, indicating strong separability between the spam and ham classes across decision thresholds. The curve rises sharply toward the top-left corner, meaning a high true positive rate can be achieved while keeping the false positive rate low, which is desirable for a spam filter where incorrectly blocking legitimate email is costly.

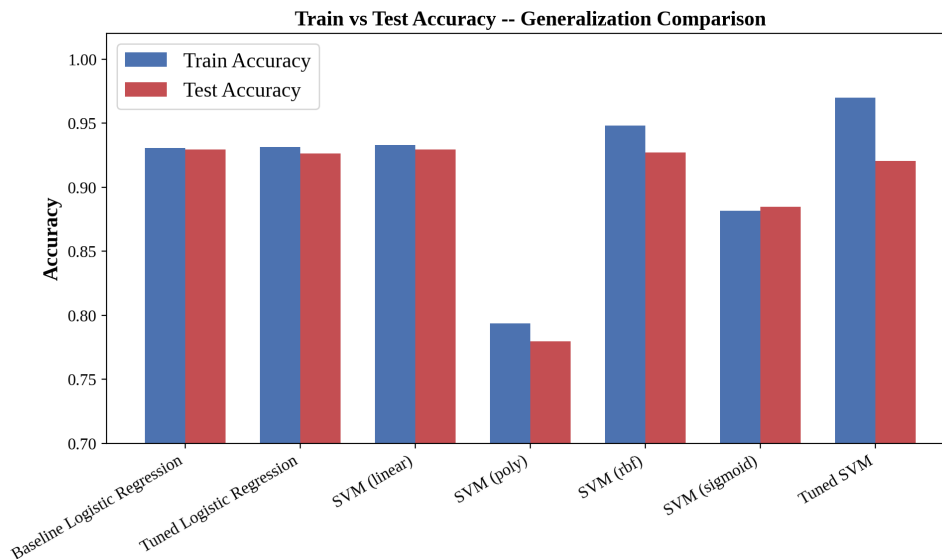


Figure 18: Train vs Test accuracy across all evaluated models (generalization comparison).

Inference: Models with a small gap between train and test accuracy (e.g. the baseline and linear-kernel models) generalize well and show low variance. The tuned SVM shows the largest train-test gap (0.0494), consistent with its higher-capacity RBF kernel fitting the training data more closely and being somewhat more prone to overfitting than the linear models, despite having the highest cross-validated training accuracy of all models evaluated. This illustrates the classical bias-variance trade-off between model flexibility and generalization.

12 Comparative Analysis

Table 7: Comparative Analysis: Logistic Regression vs. SVM (Tuned Models)

Criterion	Logistic Regression	SVM
Accuracy (test set)	92.62%	92.07%
Model Complexity	Low – linear decision boundary with one coefficient per feature ($O(d)$ parameters)	Higher – kernel trick enables non-linear decision boundaries; complexity scales with the number of support vectors
Training Time (s)	0.4291	0.1513
Interpretability	High – coefficients map directly to signed log-odds contributions per feature	Lower – decision boundary defined implicitly via support vectors and kernel similarity, harder to interpret directly (especially for non-linear kernels)

13 Observations

Best-performing Classifier

Across all models evaluated, **Baseline Logistic Regression** achieved the highest test accuracy of 92.94%. Notably, the plain linear-kernel SVM achieved an identical test accuracy (92.94%), confirming that the Spambase feature space is close to linearly separable, and that the additional representational power of non-linear kernels is not required to achieve strong performance on this dataset.

Effect of Logistic Regression Regularization and C

GridSearchCV over `penalty` $\in \{L1, L2\}$ and $C \in \{0.01, 0.1, 1, 10, 100\}$ selected **L1** regularization with **$C = 100$** . L1 regularization drives less-informative feature coefficients toward exactly zero (implicit feature selection), which is well suited to Spambase's many weakly-informative word-frequency columns. The large selected C value (weak regularization strength) indicates that, on this dataset, allowing the model more freedom to fit the training data yielded the best cross-validated accuracy of 92.66% without materially harming generalization: the tuned model's train-test accuracy gap is only 0.0054.

Effect of Solver

The grid search selected **liblinear** as the best-performing solver. `liblinear` is a coordinate-descent solver well suited to smaller feature counts (57 features here) and supports both L1 and L2 penalties efficiently, whereas `saga` is designed for larger-scale problems and, while equally capable of optimizing an L1 penalty, converges more slowly on a dataset of this size, as observed during experimentation.

SVM Kernel Behaviour

The four kernels showed markedly different baseline (untuned) behaviour: Linear reached 92.94%, RBF reached 92.73%, Sigmoid reached 88.49%, while Polynomial trailed at only 77.96% with a notably low recall of 46.01%. This is consistent with the near-linear separability of the Spambase feature space (word/character frequencies): kernels that add unnecessary non-linear flexibility without tuning (particularly the default-degree polynomial kernel) can under-fit the positive (spam) class, while the sigmoid kernel is highly sensitive to the input scale and default `gamma`, and under-performs both linear and RBF as a result.

Effect of Hyperparameter Tuning on SVM

GridSearchCV over `kernel`, $C \in \{0.1, 1, 10, 100\}$, `gamma` $\in \{\text{scale}, \text{auto}\}$, and `degree` $\in \{2, 3, 4\}$ selected **kernel=rbf**, **$C=10$** , **gamma=scale**, **degree=2**, with a best cross-validated accuracy of 93.61% – the highest CV accuracy of any configuration searched, for either model family. However, on the held-out test set the tuned SVM scored 92.07%, slightly below the untuned linear-kernel SVM and the baseline Logistic Regression, illustrating that the configuration with the best cross-validated training-fold accuracy does not always yield the best held-out generalization, particularly for a higher-capacity RBF kernel at a non-trivial C .

Training vs Test Performance and Bias-Variance Trade-off

Table 6 (Section 11) reports the train-test accuracy gap for every model. The simplest models – baseline Logistic Regression (gap 0.0013), linear-kernel SVM (gap 0.0035), and even the sigmoid kernel (gap -0.0031, slightly negative, meaning test accuracy exceeded train accuracy on this split) – show the smallest gaps, indicating low variance and good generalization. The tuned SVM shows the largest gap (0.0494), reflecting higher variance from its more flexible RBF decision boundary. This is a textbook illustration of the bias-variance trade-off: added flexibility can reduce training-set bias but, if not carefully regularized, increases variance and can slightly reduce test-set generalization.

Model Complexity and Interpretability

Logistic Regression is a low-complexity, linear, and highly interpretable model: each of its 57 coefficients has a direct, signed effect on the predicted log-odds of an email being spam, making it straightforward to explain individual predictions to a non-technical stakeholder. SVM, in contrast, is higher-complexity and less directly interpretable, especially with non-linear kernels, because its decision boundary depends on the learned support vectors and an implicit similarity function in a transformed feature space rather than on simple additive coefficients. This trade-off – interpretability and speed versus representational flexibility – is the central practical distinction between the two model families observed in this experiment.

14 Learning Outcomes

- Understanding probabilistic (Logistic Regression) classifiers and the role of the logistic/sigmoid link function in mapping linear scores to class probabilities.
- Understanding margin-based (SVM) classifiers and the kernel trick for handling non-linear decision boundaries.
- Applying systematic hyperparameter tuning via GridSearchCV with stratified cross-validation.
- Evaluating classification models using accuracy, precision, recall, F1 score, ROC curves, and AUC.
- Interpreting experimental results in terms of the bias-variance trade-off, model complexity, and interpretability.

15 Conclusion

This experiment implemented and rigorously compared Logistic Regression and Support Vector Machine classifiers for spam email detection on the UCI Spambase dataset (4601 emails, 57 numeric features, 0 missing values). Following an 80:20 stratified train-test split and feature standardization inside a scikit-learn `Pipeline` (to avoid data leakage), a baseline Logistic Regression achieved 92.94% test accuracy. GridSearchCV hyperparameter tuning over penalty, C , and solver selected penalty=11, $C=100$, solver=liblinear, yielding a best cross-validated accuracy of 92.66%. All four required SVM kernels (linear, polynomial, RBF, sigmoid) were evaluated, with test accuracies ranging from 77.96% (polynomial) to 92.94% (linear); a further

GridSearchCV over kernel, C , gamma, and degree selected kernel=rbf, $C=10$, gamma=scale, degree=2 as the best configuration by cross-validated accuracy (93.61%), though its held-out test accuracy (92.07%) was marginally lower than the untuned linear-kernel SVM. The overall best-performing classifier on the held-out test set was **Baseline Logistic Regression** at 92.94% accuracy. Five-fold cross-validation of the final tuned models confirmed the stability of these results, with mean CV accuracies of 92.66% (Logistic Regression) and 93.61% (SVM). Overall, the experiment demonstrates that for the Spambase dataset – whose features are close to linearly separable – a well-tuned linear model can match or exceed the performance of a kernel-based model, while offering substantially greater interpretability and lower computational cost; kernel methods remain valuable for capturing non-linear structure but require careful hyperparameter tuning to realize a genuine advantage over a strong linear baseline.

16 References

- [1] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. Logistic Regression documentation: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
- [2] Scikit-learn Developers, “Support Vector Machines,” scikit-learn User Guide. <https://scikit-learn.org/stable/modules/svm.html>
- [3] Scikit-learn Developers, “Tuning the hyper-parameters of an estimator (GridSearchCV),” scikit-learn User Guide. https://scikit-learn.org/stable/modules/grid_search.html
- [4] M. Hopkins, E. Reeber, G. Forman, and J. Suermondt, “Spambase Data Set,” UCI Machine Learning Repository, 1999. <https://archive.ics.uci.edu/dataset/94/spambase>
- [5] D. Dua and C. Graff, “UCI Machine Learning Repository,” University of California, Irvine, School of Information and Computer Sciences, 2019. <https://archive.ics.uci.edu>